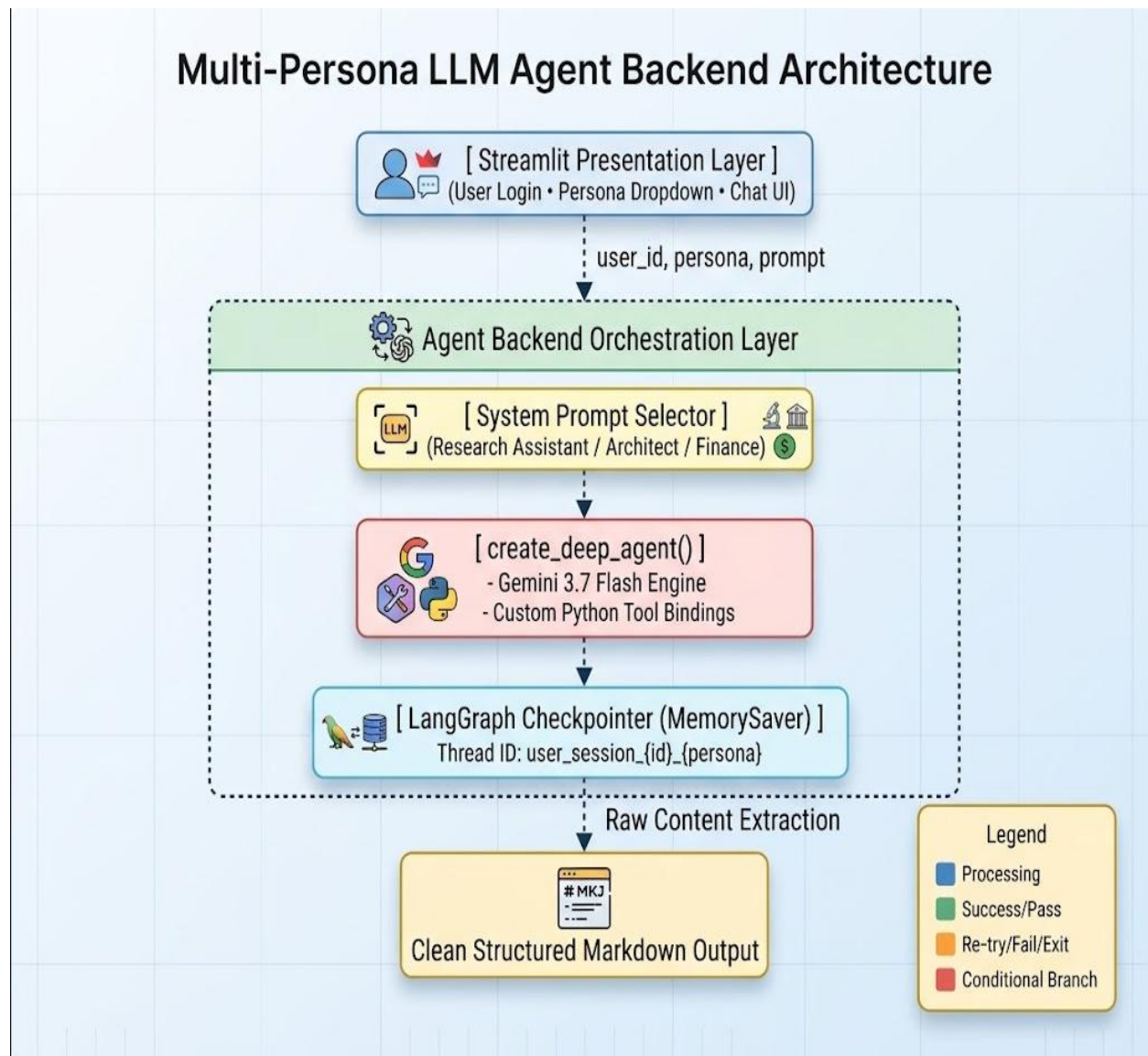


Deep Agent Multi-Persona Workspace with Stateful Persistence

System Architecture & Technical Specification

System Overview

The application is a stateful, interactive web-based agent system powered by **Deep Agents**, **LangGraph**, and Google's **Gemini 3.7 Flash** model. It decouples the presentation layer from the execution runtime, allowing users to authenticate, dynamically swap operational personas, conduct iterative tool-assisted reasoning, and retain conversation history across turns.



Component Architecture

- Frontend (app.py):** Built with Streamlit, handling authentication state, user session keys, persona switching, and chat rendering. Each session is keyed by `{user_id}_{persona_choice}` to ensure UI message logs mirror backend state.

Deep Agent Multi-Persona Workspace with Stateful Persistence

- **Orchestration & Routing (`agent_backend.py`):**
 - **Dynamic Persona Assignment:** Maps persona definitions (Research Assistant, Code Architect, Financial Analyst) directly to tailored system prompts upon initialization.
 - **Tool Call Execution:** Wraps external functions (e.g., search tools) as standard LangChain tools (`@tool`), ensuring standard schema compatibility and avoiding provider-specific server-side tool configuration conflicts.
 - **Session-Level Thread Scoping:** Calls `agent.invoke()` with a dedicated `configurable={"thread_id": ...}` namespace. Each user and persona combination receives an isolated execution graph thread.
 - **State Checkpointing:** Backed by LangGraph's checkpointing mechanism (`MemorySaver`), preserving iterative reasoning steps, tool outputs, and historical context per thread.
 - **Payload Normalization:** Traverses Gemini's structured output array, isolating text elements from raw JSON blocks and token signatures to emit clean Markdown.

Execution Flow

1. **Input Capture:** The user enters an ID, selects a persona, and submits a task.
2. **Context Compilation:** The backend fetches or instantiates the graph corresponding to `thread_id: f"user_session_{user_id}_{persona_name}"`.
3. **Reasoning Loop:** The agent evaluates the prompt against persona instructions, decides whether tool invocation is required, executes tools, and evaluates the results.
4. **State Persistence:** The updated conversational state is committed to the checkpointing snapshot.
5. **Output Parsing:** The extraction logic unpacks structured response blocks and renders the final report in the Streamlit UI.